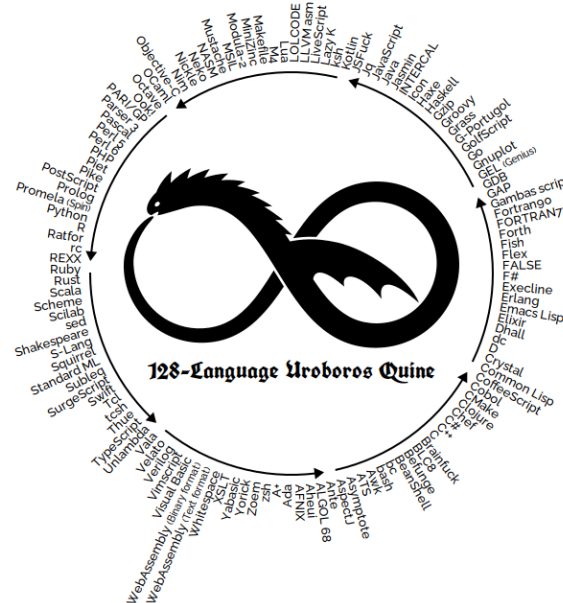


Contact

Packaging 128 languages with Nix



by Noon van der Silk



Invariant Club

[Articles](#) [Templates](#) [FAQs](#) [Contact](#)

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

The famous **quine-relay** by **@mame** project builds a so-called "uroboros" **quine** through 128 languages; i.e. from ruby to rust to scala to guile to all the way back to ruby again, where the final output ruby program is exactly equal to the original ruby program.

Clearly, this is an awesome achievement!

But, have you tried to run it yourself? If you have, you might have found it hard to reproduce. As of the time of writing, the Docker build fails. I don't run Ubuntu, so I can't tell if all the apt-get install commands work; maybe they do, but maybe not.

So, there's a natural idea. Indeed, one that **someone else had all the way back in 2021**. What if we package it with Nix? Then we can get a simple invocation, `nix build ...` that will produce the final output.

This seemed like a fun challenge, and a good test for Nix: Would we be able to do it? And if so, at what cost?

Let's get into it.

Can we do it?

Even before getting started, we know the answer to this: Yes. The **premise of including a particular language** is that it should either be available in

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

can either see if it's already on **nixpkgs**, and if not, work out how to build it from source.

Motivation high; it's now down to our willingness to do a bunch of hacking. Let's get started.

How did we do it?

Given the large package set already available through nixpkgs, the task *should* be pretty simple, and can be broken into two parts:

- Those already in Nix,
- Those we have to manually package.

Languages already in Nix

Luckily, a large portion of the languages we needed were already packaged in Nix.

Those were: ruby, rust, scala, guile, scilab, sed, slang, standard-ml, surgescript, swift, tcl, tc, typescript, vala, verilog, vim, vb, wasm, xslt, yab, zsh, ada, algol, aspectj, asymptote, ats, awk, bash, bc, beanshell, c, cpp, csharp, clojure, cmake, cobol, coffeescript, clisp, crystal, d, dhall, elixir, elisp, erlang, execline, fsharp, flex, fish, forth,

Invariant Club

[Articles](#) [Templates](#) [FAQs](#) [Contact](#)

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

lolcode, lua, m4, make, minizinc, modula2 (almost), msil, mustache, nasm, neko, nim, objectivec, ocaml, octave, pari, pascal, perl5, perl6 (raku), php, pike, postscript, prolog, spin, python, r, rc, rexx, (and ruby again!)

This is actually almost all of them: 95 languages. That leaves in principle 33 languages we have to do something a little more complicated for; in practice it's fewer as **mame** has ruby versions of interpreters for a few of the more esoteric languages.

Our approach for languages *already* in Nix was to just write a bunch of expressions ultimately of this form:

```
1  ruby = ...
2
3  rust = with pkgs; stdenv.mkDerivation {
4    src = "${ruby.out}/share";
5    nativeBuildInputs = [ rust ];
6    buildPhase = ''
7      rustc QR.rs
8      ./QR > QR.scala
9    '';
10   installPhase = ''
11     mkdir -p $out/share
12     cp QR.scala $out/share/
13
```

nix

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

While there was a little bit of extra busywork, this basic format will actually served for every step: at the n th step, look at step $n-1$ and step $n+1$ to work out what to produce.

The only question is how we get the necessary compilers/interpreters for the languages that aren't present.

Languages we had to manually package

For all but a few, the approach was this:

1. Find the source code,
2. Clone it,
3. Try and build it with Nix,
4. If that works, great!

With a little hacking here and there, adding in necessary build-time dependencies, this worked for basically all the languages aside from *gambas3* and *modula-2*.

These two proved difficult for some kind-of interesting reasons.

gambas3

Invariant Club

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

you would find that it fails at runtime.

Glancing around the sourcecode, you can find that there is a large number of references to `/usr/lib/ ...`. If you use **NixOS**, you will spot this as an immediate issue. If some program is explicitly expecting these paths to exist, they will be disappointed with prejudice.

The solution is to make use of the `buildFHSEnv` function which creates a runtime environment for packages like that to execute in.

modula-2

This one was a bit of a personal journey in learning a little bit more about Nix.

If you try and trackdown where the modula-2 sourcecode is, you find it's in **"gcc"** - the GNU compiler collection.

Great news!

In fact, that library is actually already an essential part of nixpkgs: **stdenv**, and is just a simple package on nixpkgs: **gcc on nixpkgs**.

But, if you glance at the source code for that derivation, it's quite complicated. And taking a look around, you can see that there's a place

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

At this point, I tried to make a patch to nixpkgs directly to add it. For whatever reason, it didn't work, and I discarded that idea.

Next, I tried to build *just* the m2 part of the **gcc codebase** "manually"; i.e. by writing by a Nix derivation that was "good enough". This was a mistake. The main complaint I want to make here is against monorepos. It's surprisingly hard to just isolate a small part that you want to build, even in an only-moderately complicated codebase, let alone one like gcc. I kept running into problems that were clearly addressed by all the code in nixpkgs; but I didn't want to have to spend hours trying to work out all those details.

In the end I actually found a *different* implementation of modula-2 in **ack**; and used that (briefly). This did work, and for me just highlights the benefits of smaller isolated repositories: easier to build only the thing you care about.

In the end I actually did come back to gcc's m2 through an elegant approach by **SandaruKasa** (more on Sandaru in a moment); namely, to just use the `overrideAttrs` capability to enable m2. This then works perfectly!

It was done! I could run `nix build` and get a resulting `QR.rb` file that was exactly equal to the original! 🎉

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

So I did this, and submitted my PR and it was diligently accepted? And I basked in my own brilliance and unique ability to achieve this personal goal? Not exactly.

Before I started this, I was a good citizen and I searched the PRs on the repo to see if anyone had tried; there were no results. In fact there were no open PRs at all. So fine, I rolled up my sleeves and got to work.

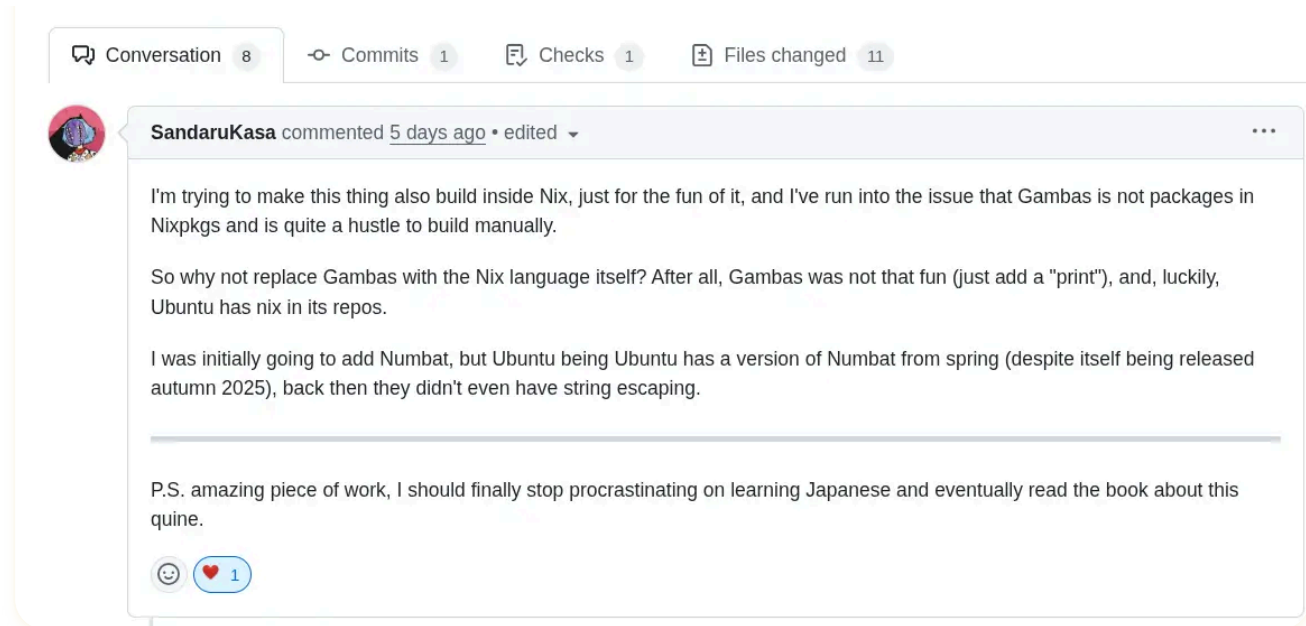
After a few late evenings of hacking, I was ready to **write up my PR**). As I was writing, I half-noticed that there was one open PR; "oh" I thought, "that's interesting, I'll have a look in a moment". I typed up my thoughts, and submitted.

I sat back, and waited for the love to roll in. "May as well see what that PR is about" I thought. **I clicked it**, and found ...

Invariant Club

[Articles](#) [Templates](#) [FAQs](#) [Contact](#)

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

!

@**SandaruKasa** had the exact same idea as me at basically the exact same time, and submitted their PR a few days before me!

I started taking a close look. Sandaru resolved many similar problems to me; often in a more elegant way. Both our approaches **worked**, and indeed we could both take interesting ideas from each others approach.

I was actually quite excited by this. First, I was very happy to learn from Sandaru, and have since refactored and improved my approach. Secondly, it also demonstrates the "ease" of packaging things with Nix: there was

Invariant Club

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

achieve the outcome in a variety of ways!

For me this is the best possible outcome.

Concluding thoughts

So is that it? Is Nix amazingly perfect and should be immediately adopted for every programming language? Maybe.

But I will note that there is a world of difference between simply *running* a programming languages compiler or interpreter; and doing every-day incremeintal development *and* keep a maintainable Nix setup that works for everyone in your team. Some programming language ecosystems are better set up with Nix than others in this sense.

But the good news, is that all of this can be improved. And the benefits you get are huge!

Appendix

Lessons learned

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

requirement for an `*nix` operating-system to have `/usr/lib/...` - yet a lot of applications assume that it (and similar paths) will exist - this is a very convenient tool for wrapping those programs so that they can still execute.

- `overrideAttrs` and **overriding in general**

Sandaru used this beautifully to bring in `gm2` from the already-heavily-configured **gcc package**.

This quite trivially solved a very complex problem: How to build a very large and complex package? Luckily, in the case of such a popular package, as we've seen, there's a very good chance it's already in `nixpkgs`!

- If something doesn't work, check another version on **nixpkgs**

Often, if something in whatever commit/branch of `nixpkgs` you've decided to use doesn't work, you may find it works in a more recent or older version.

This was true here for Swift.

Luckily, it's a very simple matter to just try a different version; by adding a new flake input: `nixpkgs2505.url = "github:nixos/nixpkgs/nixos-`

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

This is perhaps minor, but useful to remember: in your derivations you can set `enableParallelBuilding` which will then allow Nix to ... build whatever it can in parallel. Depending on your computer, this can *wildly* speed up your iteration process!

- Patching is amazing ❤️

the `mkDerivation` function allows you to supply a list of patches (i.e. `git diff ... outputs`) that it can apply to the source tree. This is a very convenient strategy for quickly getting a build that works. In this particular process, it was important to be making progress; so my technique was to patch wildly, and then go back and refine the patch (in almost all cases entirely replace it with `sed` -style replacements) once the entire build was complete.

This is a nice incremental-kind of refactoring that keeps you productive and pragmatic, but allows room for precision when you need it.

- Open source is amazing!

In fact, quite clearly, patching would be very hard if the code wasn't open-source. Even though I've worked as a programmer for over 25 years now, I still found myself amazed at how useful it was to be able to

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

- Monorepo's can be frustrating

Of course monorepos are controversial, and they can be both extremely pragmatic and productive. Myself personally I'm open-minded.

But, while hacking on this I noticed that, if you just want *part* of a monorepo, they can be extremely inconvenient; because, unless it's done *extremely well*, as an external consumer, you need to take into your head the whole structure of all the parts you *don't* care about; i.e. you need to work out what depends on what, and how to exclude the things you don't care about.

This, at least, was *not* an aspect of monorepo's that I had personally previously considered.

- Why use stable nixpkgs?

One thing that occurred to me recently is that, when working on many projects (or perhaps your company has 100s of repositories), it pays off big to use a stable input for nixpkgs ; i.e. `nixpkgs.url =`

`"github:nixos/nixpkgs/nixos-25.11` . This allows for the most re-use of build outputs between all your various projects.

- **Fixed-output derivations** (FODs) can be useful

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

difference next time.

But what if you need to access the internet? To download a package? (or even just source code?! which we do all the time.)

The solution to this is simple and elegant: just declare the hash of the contents you are expecting to receive! Nix then permits the derivation to access the network, but will error out if the resulting hash doesn't match.

This is useful for package managers that don't allow you to disable checks to the internet; or for packaging steps that require downloading some dependencies.

The code

My changes are easiest to browse here: **[silky/quine-relay/nix](#)**.

There you can find:

- the `flake.nix` entrypoint,
- `pkgs.nix` which has all the Nix derivations for the packages that weren't in `nixpkgs`
- `outputs.nix` that has each step of the quine expressed as a simple call, ultimately, to `mkDerivation` ...

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

```
nix build github:silky/quine-relay/nix
```

Then you will find all the source files in `./result` .

Perhaps the most fun, at least for me personally, was to see what the resulting **piet program** looks like (check `./result/php-to-piet/share/QR.png` .)

Bonus content

If you feel like playing around adding some new languages, take a look at this PR: **Experiment with other languages**; you just need to look at editing `./src/code-gen.rb` and then run `rake` to generate `QR.rb` .

👋 *Hello!*

I'd be very happy to help you on your Nix journey. If you're inspired, intrigued, stuck, or otherwise keen to make some progress, then please do get in **contact** :)

Invariant Club

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)

Resources

[Articles](#)[Templates](#)[FAQs](#)[Contact](#)[Getting started with Nix \(and Flakes\)](#)[How to monorepo with Nix](#)